

```

In [1]: # =====
# COMPLETE RAG PIPELINE – Run this ONE cell to Launch Gradio
# Features: Streaming, Source Highlighting, Query Rewriting, Re-ranking
# =====

import os, gc, shutil, time, tempfile
from dataclasses import dataclass, field
from collections import defaultdict

# — Install dependencies —————
import subprocess, sys
pkgs = [
    "langchain", "langchain-openai", "langchain-community",
    "langchain-text-splitters", "faiss-cpu", "tiktoken",
    "pypdf", "sentence-transformers", "rank-bm25", "gradio",
    "openai", "crossencoder"
]
for pkg in pkgs:
    subprocess.run([sys.executable, "-m", "pip", "install", pkg, "-q"],
                    capture_output=True)
print("✅ Dependencies ready")

# — Imports —————
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_community.document_loaders import PyPDFLoader
from langchain_community.vectorstores import FAISS
from langchain_community.retrievers import BM25Retriever
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.messages import HumanMessage, AIMessage
from langchain_core.documents import Document
from sentence_transformers import CrossEncoder
import gradio as gr

print("✅ Imports done")

# — Custom EnsembleRetriever —————
class EnsembleRetriever:
    def __init__(self, retrievers, weights=None):
        self.retrievers = retrievers
        self.weights = weights or [1/len(retrievers)] * len(retrievers)

    def invoke(self, query):
        scores = defaultdict(float)
        docs_map = {}
        for retriever, weight in zip(self.retrievers, self.weights):
            for i, doc in enumerate(retriever.invoke(query)):
                key = doc.page_content
                scores[key] += weight * (1 / (i + 1))
                docs_map[key] = doc
        sorted_docs = sorted(docs_map.keys(), key=lambda k: scores[k], reverse=True)
        return [docs_map[k] for k in sorted_docs]

```

```

# — CONFIG —————
OPENAI_API_KEY = "sk-proj" # 🖱️ paste your key here

CONFIG = {
    "chunk_size"      : 1500,
    "chunk_overlap"   : 300,
    "top_k"           : 10,
    "rerank_top_k"    : 5,      # ✅ keep top 5 after re-ranking
    "embedding_model" : "text-embedding-3-small",
    "llm_model"       : "gpt-4o-mini",
    "reranker_model"  : "cross-encoder/ms-marco-MiniLM-L-6-v2", # ✅ re-ranker
}

os.environ["OPENAI_API_KEY"] = OPENAI_API_KEY

# — LLM & Embeddings —————
llm = ChatOpenAI(
    model=CONFIG["llm_model"],
    temperature=0.3,
    api_key=OPENAI_API_KEY,
    streaming=True # ✅ enable streaming
)
embeddings = OpenAIEmbeddings(
    model=CONFIG["embedding_model"],
    api_key=OPENAI_API_KEY
)

# ✅ Load cross-encoder for re-ranking
print("⌚ Loading re-ranker model...")
reranker = CrossEncoder(CONFIG["reranker_model"])
print("✅ LLM + Embeddings + Re-ranker ready")

# — Prompts —————
# ✅ Query rewriting prompt
rewrite_prompt = ChatPromptTemplate.from_messages([
    ("system", """"You are a search query optimizer.
Rewrite the user's question to be more specific and searchable for a document retrieval.
- Keep the core intent
- Expand abbreviations
- Add relevant technical terms
- Return ONLY the rewritten query, nothing else"""),
    ("human", "{question}")
])

chat_prompt = ChatPromptTemplate.from_messages([
    ("system", """"You are an expert document analyst with broad general knowledge.

RULES:
- Read ALL context chunks carefully before answering
- Even if chunks are cut off mid-sentence, piece together the full meaning
- If the answer spans multiple chunks, combine them into one complete answer
- If numbers, days, or amounts are mentioned – state them exactly as written
- If the document has partial info, provide what IS there and supplement with general knowledge
- NEVER say "I don't have enough information" – always give the best possible answer
- Format clearly with bullet points if multiple items""")
])

```

Always structure your answer as:

 From your documents:
[extract everything relevant from the PDF chunks]

 Additional context:
[your own knowledge to enrich or complete the answer, or "Nothing to add." if compl

Document Context:

```
{context}"""),
    ("placeholder", "{chat_history}"),
    ("human", "{question}")
])
```

```
general_prompt = ChatPromptTemplate.from_messages([
    ("system", """"You are a knowledgeable assistant.
No document chunks were found for this question.
Answer using your general knowledge. Be specific and helpful.
End with: "⚠️ Answer based on general knowledge, not your uploaded documents." """"),
    ("placeholder", "{chat_history}"),
    ("human", "{question}")
])
```

— Pipeline state

```
pipeline = {
    "vectorstore"      : None,
    "hybrid_retriever": None,
    "chunks"           : [],
    "loaded_files"     : [],
}
```

— Helper functions

 Query rewriting

```
def rewrite_query(question: str) -> str:
    try:
        chain = rewrite_prompt | llm | StrOutputParser()
        rewritten = chain.invoke({"question": question})
        print(f"# fallback to original if rewriting fails
```

 Re-ranking with cross-encoder

```
def rerank_docs(query: str, docs: list, top_k: int) -> list:
    if not docs:
        return docs
    try:
        pairs = [(query, doc.page_content) for doc in docs]
        scores = reranker.predict(pairs)
        scored_docs = sorted(zip(scores, docs), key=lambda x: x[0], reverse=True)
        reranked = [doc for _, doc in scored_docs[:top_k]]
        print(f"
```

```

#  Format context with source highlighting
def format_context(docs):
    parts = []
    for i, doc in enumerate(docs, 1):
        src = doc.metadata.get("source_file", "unknown")
        page = doc.metadata.get("page", "?")
        cidx = doc.metadata.get("chunk_index", "?")
        parts.append(f"--- Chunk {i} | {src} | page {page} ---\n{doc.page_content}")
    return "\n\n".join(parts)

#  Format source references shown below the answer
def format_sources(docs) -> str:
    if not docs:
        return ""
    seen = set()
    lines = ["\n\n---\n📄 **Sources:**"]
    for doc in docs:
        src = doc.metadata.get("source_file", "unknown")
        page = doc.metadata.get("page", "?")
        key = f"{src}::p{page}"
        if key not in seen:
            seen.add(key)
            lines.append(f"- 📄 `{src}` - Page {int(page)+1 if str(page).isdigit()")
    return "\n".join(lines)

def build_pipeline(pdf_paths: list, progress=None) -> str:
    global pipeline

    if not pdf_paths:
        return "❌ No PDF files provided."

    status_lines = []

    try:
        # Step 1: Load PDFs
        if progress: progress(0.1, "📄 Loading PDFs...")
        raw_docs = []
        loaded = []
        for path in pdf_paths:
            try:
                loader = PyPDFLoader(path)
                pages = loader.load()
                for page in pages:
                    page.metadata["source_file"] = os.path.basename(path)
                raw_docs.extend(pages)
                loaded.append(os.path.basename(path))
                status_lines.append(f"
```

```

splitter = RecursiveCharacterTextSplitter(
    chunk_size=CONFIG["chunk_size"],
    chunk_overlap=CONFIG["chunk_overlap"],
    separators=["\n\n", "\n", ". ", " ", ""],
)
chunks = splitter.split_documents(raw_docs)
chunk_counts = {}
for chunk in chunks:
    src = chunk.metadata.get("source_file", "unknown")
    chunk_counts[src] = chunk_counts.get(src, 0) + 1
    chunk.metadata["chunk_index"] = chunk_counts[src]
status_lines.append(f"🔪 {len(chunks)} chunks created")

# Step 3: Embed
if progress: progress(0.6, "📄 Embedding chunks (this takes ~30s)...")
gc.collect()
vectorstore = FAISS.from_documents(documents=chunks, embedding=embeddings)
status_lines.append(f"📄 {len(vectorstore.docstore._dict)} vectors embedded")

# Step 4: Build hybrid retriever
if progress: progress(0.85, "🔍 Building retriever...")
dense = vectorstore.as_retriever(search_type="similarity",
                                search_kwargs={"k": CONFIG["top_k"]})
bm25 = BM25Retriever.from_documents(chunks)
bm25.k = CONFIG["top_k"]
hybrid = EnsembleRetriever(retrievers=[dense, bm25], weights=[0.6, 0.4])

pipeline["vectorstore"] = vectorstore
pipeline["hybrid_retriever"] = hybrid
pipeline["chunks"] = chunks
pipeline["loaded_files"] = loaded

if progress: progress(1.0, "✅ Done!")

summary = "\n".join(status_lines)
return f"✅ Pipeline ready!\n\n{summary}\n\n💬 You can now ask questions b

```

```

except Exception as e:
    return f"❌ Pipeline error: {str(e)}"

```

✅ Streaming chat function with query rewriting + re-ranking + source highlighting

```

def chat_fn(question, history):
    if not question.strip():
        return history, ""

    if pipeline["hybrid_retriever"] is None:
        history.append({"role": "user", "content": question})
        history.append({"role": "assistant",
                        "content": "⚠️ Please upload a PDF first using the panel on the left."})
        yield history, ""
        return

```

```

# ✅ Step 1: Rewrite query for better retrieval
rewritten_q = rewrite_query(question)

```

```

# ✅ Step 2: Retrieve using rewritten query
retrieved_docs = pipeline["hybrid_retriever"].invoke(rewritten_q)

# ✅ Step 3: Re-rank retrieved docs
reranked_docs = rerank_docs(rewritten_q, retrieved_docs, CONFIG["rerank_top_k"])

context = format_context(reranked_docs) if reranked_docs else ""

# ✅ Step 4: Build source references
sources_text = format_sources(reranked_docs) if reranked_docs else ""

# Build LangChain history
lc_history = []
for msg in history:
    if isinstance(msg, dict):
        if msg.get("role") == "user":
            lc_history.append(HumanMessage(content=msg["content"]))
        elif msg.get("role") == "assistant":
            lc_history.append(AIMessage(content=msg["content"]))

# ✅ Step 5: Add placeholder for streaming
history.append({"role": "user", "content": question})
history.append({"role": "assistant", "content": "⌚ Thinking..."})
yield history, ""

# ✅ Step 6: Stream response
try:
    if reranked_docs:
        chain = chat_prompt | llm | StrOutputParser()
        stream = chain.stream({"context": context, "question": question,
                               "chat_history": lc_history})
    else:
        chain = general_prompt | llm | StrOutputParser()
        stream = chain.stream({"question": question, "chat_history": lc_history})

    full_answer = ""
    for token in stream:
        full_answer += token
        history[-1] = {"role": "assistant", "content": full_answer}
        yield history, ""

    # ✅ Step 7: Append sources after streaming completes
    final_answer = full_answer + sources_text
    history[-1] = {"role": "assistant", "content": final_answer}
    yield history, ""

except Exception as e:
    history[-1] = {"role": "assistant", "content": f"❌ Error: {str(e)}"}
    yield history, ""

def upload_and_process(files, progress=gr.Progress()):
    if not files:
        return "⚠️ No files uploaded.", gr.update()
    paths = [f.name for f in files] if isinstance(files[0], object) else files
    status = build_pipeline(paths, progress=progress)

```

```

loaded = pipeline["loaded_files"]
files_label = ", ".join(loaded) if loaded else "No files loaded"
return status, gr.update(value=f"📁 Loaded: {files_label}")

# — Gradio UI —————
with gr.Blocks(title="RAG Document Chat") as demo:

    gr.Markdown("# 📄 RAG Document Chat\nUpload any PDF → ask anything about it.")
    gr.Markdown("> ✨ Features: **Streaming** · **Query Rewriting** · **Re-ranking**")

    with gr.Row():

        with gr.Column(scale=1):
            gr.Markdown("### 📁 Upload Documents")
            file_upload = gr.File(
                label="Upload PDF(s)",
                file_types=[".pdf"],
                file_count="multiple"
            )
            upload_btn = gr.Button("⚙️ Process Documents", variant="primary")
            file_status = gr.Textbox(
                label="📁 Status",
                value="No files loaded yet.",
                lines=1,
                interactive=False
            )
            process_output = gr.Textbox(
                label="Processing Log",
                lines=10,
                interactive=False
            )
            gr.Markdown("### 💡 Example Questions")
            gr.Examples(
                examples=[
                    ["What is the pre hospitalization period?"],
                    ["What are the disease categories defined?"],
                    ["What is covered under day care treatment?"],
                    ["What are the exclusions in this policy?"],
                    ["What is the waiting period for pre existing diseases?"],
                    ["What is post hospitalization coverage?"],
                    ["Is mental illness covered?"],
                    ["What is the sum insured?"],
                ],
                inputs=[],
                label=""
            )

        with gr.Column(scale=2):
            gr.Markdown("### 💬 Chat")
            chatbot = gr.Chatbot(
                height=520,
                show_label=False,
                render_markdown=True # ✅ render markdown for source links
            )
            with gr.Row():

```

```

        txt = gr.Textbox(
            placeholder="Ask anything about your uploaded documents...",
            scale=4,
            show_label=False,
            lines=1
        )
        send_btn = gr.Button("Send 🚀", scale=1, variant="primary")
        clear_btn = gr.Button("🗑️ Clear Chat", variant="secondary")

    upload_btn.click(
        fn=upload_and_process,
        inputs=[file_upload],
        outputs=[process_output, file_status]
    )
    send_btn.click(
        fn=chat_fn,
        inputs=[txt, chatbot],
        outputs=[chatbot, txt],
        show_progress=False # ✅ hide spinner since we stream
    )
    txt.submit(
        fn=chat_fn,
        inputs=[txt, chatbot],
        outputs=[chatbot, txt],
        show_progress=False
    )
    clear_btn.click(fn=lambda: ([], ""), outputs=[chatbot, txt])

print("✅ Launching Gradio...")
demo.launch(theme=gr.themes.Soft())

```

✅ Dependencies ready

C:\Users\thema\AppData\Local\Temp\ipykernel_3536\1730941256.py:25: DeprecationWarning: `langchain-community` is being sunset and is no longer actively maintained. See <https://github.com/langchain-ai/langchain-community/issues/674> for details and migration guidance toward standalone integration packages.

```
from langchain_community.document_loaders import PyPDFLoader
```

✅ Imports done

⌚ Loading re-ranker model...

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

Loading weights: 0%| | 0/105 [00:00<?, ?it/s]

✅ LLM + Embeddings + Re-ranker ready

✅ Launching Gradio...

* Running on local URL: <http://127.0.0.1:7860>

* To create a public link, set `share=True` in `launch()`.



RAG Document Chat

Upload any PDF → ask anything about it.

✦ Features: **Streaming** · **Query Rewriting** · **Re-ranking** · **Source Highlighting**

📁 Upload Documents

📄 Upload PDF(s)



Drop File Here

- or -

Click to Upload

📄 Upload Documents

Out[1]:

In []: